Teoría de la computación
Noviembre , 2025

Laboratorio 12

Para este laboratorio cree un repositorio de github

- Grabe un video de no más de 10 minutos donde muestre la ejecución de su programa para el ejercicio1. Súbalo a YouTube como video no listado y adjúntelo en el README. de su repositorio.
- Para este ejercicio utilice cualquier otro lenguaje que permita programación funcional que no sea python.
- Para los incisos que no involucren la generación de código, cree una carpeta por separado en su repositorio y coloque las respuestas en un documento con formato PDF

~~Problema 1: 25%~~

- Escriba la reducción-β de la operación lógica NOT
- Escriba y explique como se vería la recursión y los ciclos.
- Explique cuando es prudente usar este tipo de programación y cuando no. De un ejemplo para cada caso.

Problema 2: 25%

Escribir un programa que ordene una lista de diccionarios con respecto a un `key` indicado, usando funciones lambda.

Ejemplo

Si la lista original de diccionarios es:

```
[
    {'make': 'Nokia', 'model': 216, 'color': 'Black'},
    {'make': 'Mi Max', 'model': 2, 'color': 'Gold'},
    {'make': 'Samsung', 'model': 7, 'color': 'Blue'}
]
```

La salida (si ordenamos con respecto a `key = 'model'`) debe ser:

```
[
    {'make': 'Mi Max', 'model': 2, 'color': 'Gold'},
    {'make': 'Samsung', 'model': 7, 'color': 'Blue'},
    {'make': 'Nokia', 'model': 216, 'color': 'Black'}
]
```

Problema 1: 25%

- Escriba la reducción-β de la operación lógica NOT
- Escriba y explique como se vería la recursión y los ciclos.
- Explique cuando es prudente usar este tipo de programación y cuando no. De un ejemplo para cada caso.

1. Reducción de NOT

$V = \lambda a.\ \lambda b.\ a$

$F = \lambda a.\ \lambda b.\ b$

$NOT = \lambda p.\ p\ F\ V$

negación de verdadero

$NOT\ V = (\lambda p.\ p\ F\ V)\ V$

$\rightarrow_\beta (V\ F)\ V$

$= ((\lambda a.\ \lambda b.\ a)\ F)\ V$

$\rightarrow_\beta (\lambda b.\ F)\ V$

$\rightarrow_\beta F$

negación de falso

$NOT\ F = (\lambda p.\ p\ F\ V)\ F$

$\rightarrow_\beta (F\ F)\ V$

$= ((\lambda a.\ \lambda b.\ b)\ F)\ V$

$\rightarrow_\beta (\lambda b.\ b)\ V$

$\rightarrow_\beta V$

2. Recursión y ciclos

recursión

Utizaremos factorial para visualizar la recursión. El caso base es $0! = 1$. En los demás casos, multiplicamos n por el factorial de n-1. En cálculo lambda, el combinador Y nos permite expresar esta llamada recursiva tal que:

$G = \lambda r.\ \lambda n.\ SI\ (ES_CERO\ n)\ 1\ (MULT\ n)\ (r\ (PRED\ n)))$

$Y = \lambda g.\ (\lambda x.\ g\ (x\ x))\ (\lambda x.\ g\ (x\ x))$

$FACT = Y\ G$

$FACT\ 3 = 3 \times 2 \times 1 \times FACT\ 0 = 6$

PRED resta uno y MULT multiplica. Cuando llega a cero, SI elige 1 y termina la recursión.

ciclos

El numeral de church TRES aplica una operación tres veces. Cada vuelta usa el resultado de la anterior. Si SUC suma uno y empezamos en cero, obtenemos $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ Tal que:

$TRES = \lambda f.\ \lambda x.\ f\ (f\ (f\ x))$

$TRES\ SUC\ 0 \rightarrow_\beta^* SUC\ (SUC\ (SUC\ 0)) = 3$

3. ¿Cuándo conviene utilizar programación funcional?

Conviene usarla cuando queremos transformar datos con operaciones como filtrar, ordenar o convertir valores. Por ejemplo, podemos filtrar un inventario para obtener los productos disponibles y conservar la lista original para otros cálculos.

Cuando necesitamos controlar el uso de memoria, puede convenir más un ciclo que modifique los datos directamente. Por ejemplo: para ajustar el brillo de un video en tiempo real podemos actualizar los pixeles en un búfer reutilizable y evitar una copia completa por cada cuadro.

Teoría de la computación
Noviembre , 2025

Problema 3: 25%

Se tiene una matriz X (definida como lista de listas). Escribir un programa que calcule la matriz transpuesta X^T, usando funciones lambda. La salida debe darse también como una lista de listas.

Ejemplo

Si la matriz original es:

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Entonces la salida es:

$$X^T = \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$

Problema 4: 25%

Escribir un programa que elimine elementos indicados de una lista, usando funciones lambda.
Los elementos a borrar deben indicarse como una lista, y la salida debe ser la lista de elementos remanentes.

Ejemplo

Si la lista inicial es:

```
['rojo', 'verde', 'azul', 'amarillo', 'gris', 'blanco', 'negro']
```

Y la lista de elementos a borrar es:

```
['amarillo', 'cafe', 'blanco']
```

Entonces la salida debe ser:

```
['rojo', 'verde', 'azul', 'gris', 'negro']
```

Nota: Observe que 'café' no estaba en la lista original, por lo que simplemente se ignora.